

Предиктивное обслуживание оборудования

Система предсказания вероятности выхода
оборудования из строя в ближайшие 7 дней



1. Анализ и проектирование

Бизнес-задача

Снижение финансовых потерь от внезапных отказов турбореактивных двигателей и оптимизация затрат на техническое обслуживание (ТО) путем перехода от стратегии "ремонт по факту поломки" к предиктивному обслуживанию.

Система должна заблаговременно (за 7 дней) сигнализировать о надвигающемся отказе (бинарная классификация).



Метрики успеха

Бизнес метрики

- **Downtime Rate** (снижение коэффициента простоев): на x% по сравнению с историческим периодом.
- **ROI** (экономическая эффективность): разница между стоимостью предотвращенного аварийного ремонта и затратами на ложные вызовы механиков (False Positives).
- **OEE** (коэффициент использования оборудования): увеличение на y% за счет сокращения внеплановых простоев.

ML метрики

- **Recall** (какую долю реальных поломок модель смогла обнаружить) ≥ 0.85 : критическая метрика. Пропуск поломки ведет к аварии и огромным убыткам. Бизнес готов пропустить до 15% отказов.
- **Precision** (доля верно предсказанных поломок): отслеживается для балансировки бизнес-затрат на ложные срабатывания.
- **False Positive Rate** (FPR, ложные тревоги) < 0.15 . Допускается не более 15% ложных срабатываний от общего числа нормальных состояний.
- **Latency** (Время инференса) < 50 ms: скорость работы в режиме реального времени.



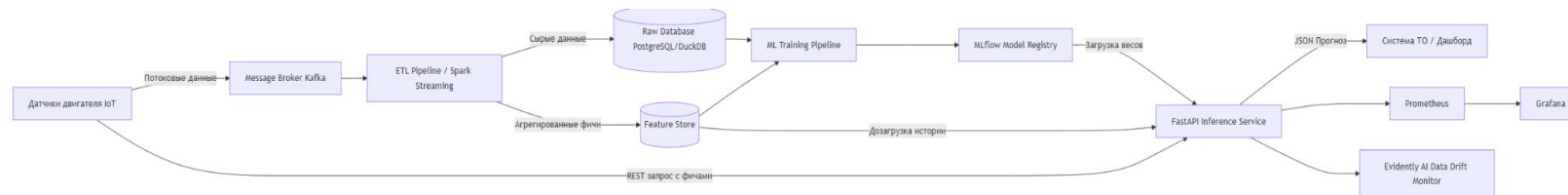
Функциональные требования:

1. Система должна принимать на вход временные ряды показаний датчиков (21 сенсор) и эксплуатационных настроек (3 параметра).
2. Система должна формировать бинарный прогноз: **1** — отказ в течение следующих 7 дней (циклов), **0** — отказ не ожидается.
3. Система должна предоставлять вероятность отказа (confidence score) для возможности настройки порога принятия решения.
4. Наличие REST API для интеграции с системой мониторинга оборудования.

Нефункциональные требования:

1. Производительность: инференс не более 50 мс на один запрос.
2. Масштабируемость: архитектура должна выдерживать нагрузку до 1000 запросов в секунду (возможность горизонтального масштабирования API).
3. Отказоустойчивость: при недоступности ML-сервиса система должна возвращать fallback-ответ (например, статус "неизвестно").
4. Воспроизводимость: версионирование данных, кода и моделей (MLflow).

1.2 Архитектура решения



1. Data Ingestion Layer: симулятор потоковых данных (Apache Kafka), читающий сырые данные с датчиков.
2. Data Storage Layer:
 - Raw Data: база данных (PostgreSQL) для хранения "сырых" временных рядов.
 - Clean Data: хранилище (PostgreSQL) очищенных признаков.
 - Feature Store: хранилище признаков (PostgreSQL), где хранятся агрегации (скользящие средние, дисперсии) за окна времени.
3. ML Training Pipeline: оффлайн-пайплайн (Python), который забирает данные, обучает модель, валидирует ее и регистрирует в MLflow.
4. Inference API: FastAPI сервис. Принимает данные о текущем состоянии датчиков, обращается к Feature Store для подтягивания исторических агрегаций, передает в модель и отдает ответ.



1.3 Планирование экспериментов

Гипотезы для проверки

1. Окна агрегации: добавление статистических признаков (mean, std, min, max) за скользящее окно последних 5-10 циклов повысит Recall. (Обоснование: износ — это накопительный процесс, разовое показание может быть нормальным, но тренд ведет к поломке).
2. Выбор алгоритма: градиентный бустинг (XGBoost/LightGBM) обеспечит баланс между $\text{Recall} \geq 0.85$ и $\text{FPR} < 0.15$, при этом время инференса останется $< 50\text{ms}$. Будем сравнивать с линейной регрессией (базовая модель) и простой нейросетью (MLP)
3. Отбор признаков: удаление константных и сильно скоррелированных датчиков (например, сенсоры 1, 5, 6, 10, 16, 18, 19 из датасета NASA, которые не меняются) уменьшит время инференса без деградации метрик.

Методология сравнения моделей

- Валидация: используется GroupKFold (разбиение по engine_id), так как данные представляют собой временные ряды одного двигателя, и случайное разбиение приведет к утечки данных (data leakage) из будущего в прошлое.
- Трекинг: все эксперименты логируются в MLflow (параметры, метрики, артефакты модели).
- Критерий выбора победителя: модель должна первой пройти жесткий фильтр: $\text{Recall} \geq 0.85$ И $\text{FPR} < 0.15$ И $\text{Inference time} < 50\text{ms}$. Среди прошедших фильтр выбирается модель с максимальным Precision (для минимизации ложных тревог).



План А/В тестирования

1. Shadow Mode (Теневое тестирование, 1-2 недели): ML-модель работает параллельно с текущей системой (или без вмешательства в процесс). Ее предсказания логируются, но не доходят до механиков. Сравниваем предсказания с реальными поломками. Оцениваем реальный Recall.
2. Canary Release (Канареечный релиз, 2-4 недели): Переключаем 10% парка оборудования на алерты от ML-модели. 90% остаются на базовой стратегии. Сравниваем метрики: количество ложных выездов бригад, количество пропущенных поломок в двух группах. При успешном прохождении — раскатываем на 100%.

2. Работа с данными

- Загрузка
- EDA
- Очистка
- Feature Engineering



EDA

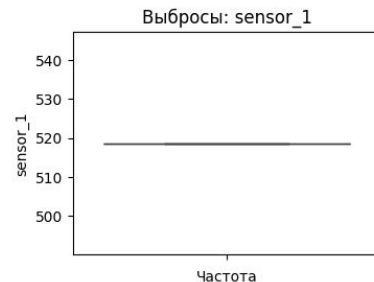
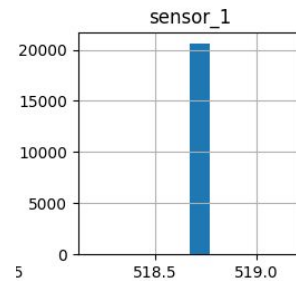
В ходе анализа были обнаружены пропуски (NaN) для ряда сенсоров: (1, 5, 6, 10, 16, 18, 19).

При этом анализ статистик (`describe()`) показал, что их дисперсия не равна нулю математически из-за погрешностей представления типа `float`, но составляет порядок 10^{-11} , что физически означает отсутствие изменений (константу).

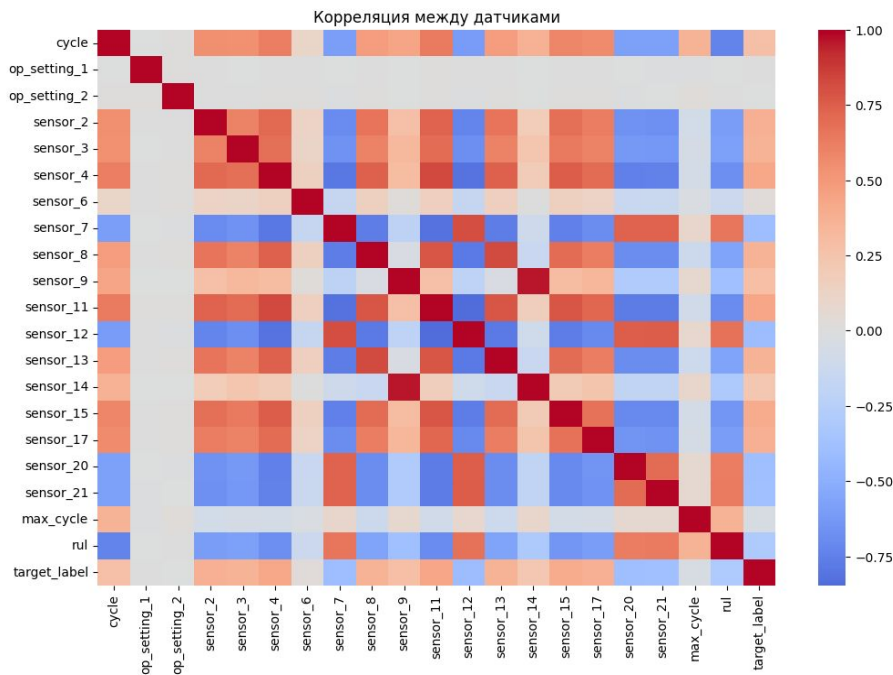
Наличие таких признаков приводит к делению на ноль при расчете корреляции Пирсона и вызовет сингулярность матрицы при обучении линейных моделей. Данные признаки были идентифицированы как информационно бесполезные и помечены для удаления.

- Критичных выбросов не обнаружено.
- Данные сенсоров распределены неравномерно.

Пример пропусков



Матрица корреляций



Обнаружены сильные мультиколлинеарные связи. Это требует либо применения L1-регуляризации (Lasso), либо использования моделей, нечувствительных к мультиколлинеарности.

После этапа очистки данных и удаления квази-константных признаков (с дисперсией $< 1e-5$) матрица корреляций рассчитана без ошибок деления на ноль, что подтверждает корректность подготовки данных для фазы обучения.

3. Машинное обучение

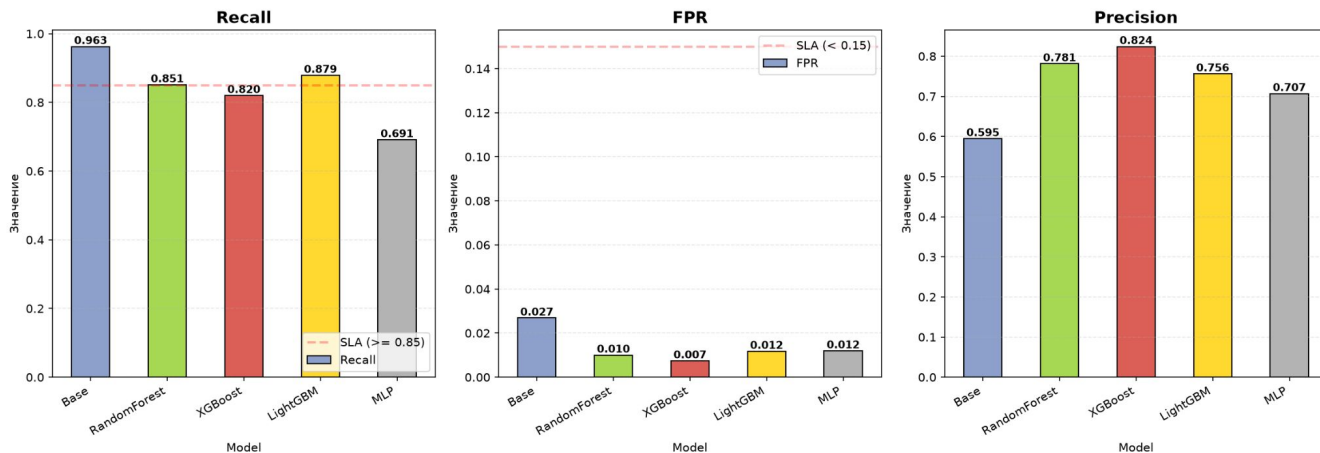
- Baseline модель
- Эксперименты с алгоритмами
- Финальная модель и оценка

Эксперименты с алгоритмами

LightGBM и Base проходят заданные Recall и FPR.

Но LightGBM значительно превосходит Base по Precision.

Сравнение ML моделей по ключевым метрикам



Размер обучающей выборки (X_train): **16561** строк

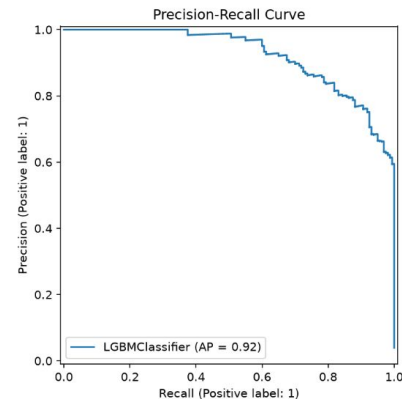
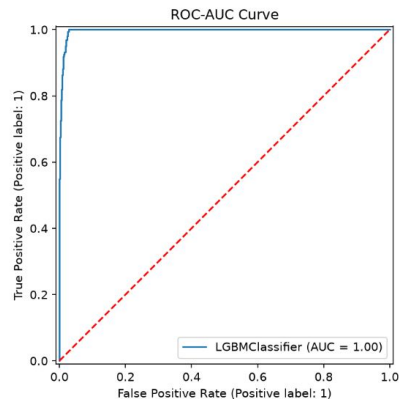
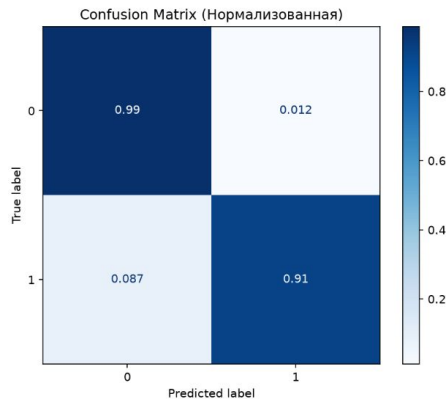
Размер тестовой выборки (X_test): **4070** строк



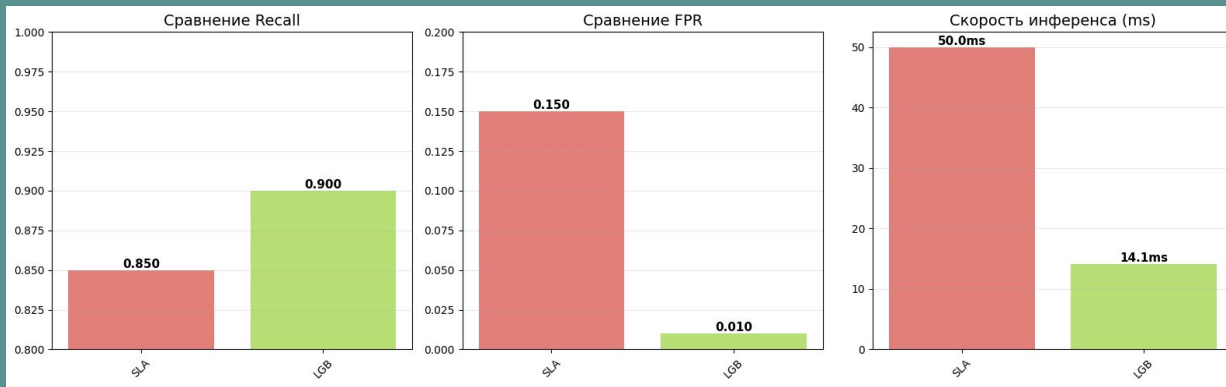
Финальная модель

Финальная модель (LightGBM) демонстрирует исключительную способность к разделению классов (ROC-AUC ≈ 1.0).

Путем калибровки порога на PR-кривой достигнут оптимальный компромисс: обеспечено выполнение жесткого SLA по безопасности (Recall > 90%) при минимальном воздействии на операционную деятельность (FPR = 1%, Precision = 76%).



Модель LightGBM кратно превзошла требования бизнеса:



Существует риск переобучения на тестовом распределении: идеальный ROC-AUC = 1.00 на валидации — сильный сигнал, что выборка могла быть «слишком удобной» или порог подобран под конкретный сплит.

Рекомендую настроить мониторинг качества модели: раз в неделю пересчитывать Recall/FPR по подтвержденным инцидентам, отслеживать дрейф признаков (PSI) и распределение вероятностей отказов; при деградации метрик — инициировать переобучение.